

Faire tourner NaileR : un exemple reproductible

Interruption de jeu

Pourquoi cette interruption ?

Cette fiche montre comment faire tourner NaileR sur un exemple complet, reproductible et indépendant du plateau.

Dans le plateau, on a déjà construit des prompts à la main à partir de sorties comme `condes()` ou `catdes()`. L'objectif de cette fiche est de montrer que NaileR peut automatiser cette logique.

L'objectif n'est pas encore d'appeler un modèle de langage. L'objectif est de vérifier que l'on sait produire un prompt ou un artefact inspectable.

```
données simulées
→ description statistique
→ sortie FactoMineR
→ fonction NaileR
→ prompt ou artefact contrôlé
→ génération optionnelle
```

Le principe à retenir

NaileR ne remplace pas FactoMineR.

Il intervient **après** la production ou la préparation d'une sortie statistique, pour organiser une demande d'interprétation plus contrôlée.

Le point central est donc :

```
Je ne demande pas au modèle d'interpréter directement les données brutes.
Je construis d'abord un objet statistique, puis un prompt ou un artefact inspectable.
```

L'option clé est :

```
generate = FALSE
```

Elle permet de préparer le prompt ou l'artefact sans appeler de modèle de langage.

Installer ou charger les packages

Cette fiche utilise **FactoMineR** pour produire les sorties descriptives et **NaileR** pour construire les prompts ou artefacts.

```
# À faire une seule fois si nécessaire
install.packages("FactoMineR")

# Si NaileR est installé depuis GitHub
install.packages("remotes")
remotes::install_github("Sebastien-Le/NaileR")
```

Puis on charge les packages.

```
library(FactoMineR)
library(NaileR)
```

Vérifier les fonctions disponibles

Selon la version installée, les fonctions disponibles peuvent varier. On peut inspecter les fonctions exportées par le package.

```
fonctions_nailer <- grep(
  "^nail_",
  getNamespaceExports("NaileR"),
  value = TRUE
)

fonctions_nailer
```

Cette étape est importante : elle distingue le support de cours de l'environnement R réellement disponible sur la machine.

Créer un petit jeu de données

On crée un jeu de données simulé.

Chaque ligne représente un répondant. On dispose de variables quantitatives, de variables qualitatives et d'un court commentaire libre.

```
set.seed(123)

n <- 120

questionnaire_demo <- data.frame(
  satisfaction = round(rnorm(n, mean = 6.2, sd = 1.2), 1),
  intention_achat = round(rnorm(n, mean = 6.0, sd = 1.5), 1),
```

```

prix_percu = round(rnorm(n, mean = 5.5, sd = 1.4), 1),
naturalite = round(rnorm(n, mean = 6.0, sd = 1.1), 1),
confiance = round(rnorm(n, mean = 5.8, sd = 1.2), 1),
type_produit = factor(
  sample(
    c("Classique", "Végétal", "Local"),
    n,
    replace = TRUE
  )
),
budget_constraint = factor(
  sample(
    c("Faible", "Moyen", "Fort"),
    n,
    replace = TRUE
  )
),
profil_alim = factor(
  sample(
    c("Pragmatique", "Curieux", "Engagé"),
    n,
    replace = TRUE
  )
)
)

commentaires_possibles <- c(
  "Produit agréable mais un peu cher.",
  "J'aime le côté naturel et simple.",
  "Le goût est correct mais pas assez gourmand.",
  "Je pourrais l'acheter si le prix reste raisonnable.",
  "Le produit me semble bon pour une consommation régulière.",
  "Je suis intéressé par l'origine locale.",
  "Je ne vois pas vraiment la différence avec un produit classique.",
  "Le produit est rassurant mais manque un peu de plaisir."
)

questionnaire_demo$commentaire <- sample(
  commentaires_possibles,
  n,
  replace = TRUE
)

head(questionnaire_demo)

```

Ce jeu de données n'a pas vocation à être réaliste. Il sert à produire des sorties manipulables.

Exemple 1 : décrire une variable qualitative avec catdes()

On commence par une variable qualitative explicite : `profil_alim`.

L'objectif est de caractériser les groupes de répondants selon les autres variables du tableau.

```
num_var_catdes <- which(
  names(questionnaire_demo) == "profil_alim"
)

num_var_catdes
```

On utilise ensuite `FactoMineR::catdes()`.

```
res_catdes <- FactoMineR::catdes(
  questionnaire_demo,
  num.var = num_var_catdes
)

names(res_catdes)
```

L'objet `res_catdes` contient une description statistique de la variable qualitative. On peut l'inspecter directement.

```
res_catdes
```

Cette sortie est riche, mais elle n'est pas encore un prompt. `NaileR` intervient pour transformer cette sortie en objet interprétatif plus contrôlé.

Produire un prompt avec nail_catdes()

L'appel le plus important est le suivant.

```
res_nail_catdes <- NaileR::nail_catdes(
  questionnaire_demo,
  num.var = num_var_catdes,
  generate = FALSE
)

res_nail_catdes
```

Le rôle de `generate = FALSE` est central : on demande à `NaileR` de préparer le prompt ou l'artefact, mais pas d'appeler un modèle de langage.

Si l'objet retourné contient un champ `prompt`, on peut l'extraire.

```

if (is.list(res_nail_catdes) && "prompt" %in% names(res_nail_catdes)) {
  texte_prompt_catdes <- res_nail_catdes$prompt
} else {
  texte_prompt_catdes <- paste(
    capture.output(print(res_nail_catdes)),
    collapse = "\n"
  )
}

cat(substr(texte_prompt_catdes, 1, 1500))

```

Le prompt devient un objet inspectable.

Exemple 2 : décrire une variable quantitative avec `condes()`

On peut aussi décrire une variable quantitative. Ici, on choisit `intention_achat`.

```

num_var_condes <- which(
  names(questionnaire_demo) == "intention_achat"
)

num_var_condes

```

On utilise `FactoMineR::condes()`.

```

res_condes <- FactoMineR::condes(
  questionnaire_demo,
  num.var = num_var_condes
)

names(res_condes)
res_condes

```

`condes()` décrit une variable quantitative à partir des autres variables du tableau. Selon la nature des variables, la logique mobilisée n'est pas la même :

```

quantitative  quantitative : corrélations
qualitative   quantitative : comparaisons de moyennes

```

Produire un prompt avec `nail_condes()`

Si la fonction `nail_condes()` est disponible dans la version installée de NaileR, on peut l'utiliser de manière analogue.

```
if ("nail_condes" %in% getNamespaceExports("NaileR")) {

  res_nail_condes <- NaileR::nail_condes(
    questionnaire_demo,
    num.var = num_var_condes,
    generate = FALSE
  )

  res_nail_condes
}
```

Si la fonction n'est pas disponible, on peut vérifier les fonctions `nail_*` exportées.

```
grep(
  "^nail_",
  getNamespaceExports("NaileR"),
  value = TRUE
)
```

Inspecter les options disponibles

Les options exactes dépendent de la fonction et de la version du package.

On peut les inspecter avec `formals()`.

```
formals(NaileR::nail_catdes)
```

Cette commande permet de vérifier si la fonction accepte des arguments comme :

- `generate` ;
- `model` ;
- `llm_model` ;
- `llm_engine` ;
- `interpretation_mode` ;
- `drop.negative` ;
- `isolate.groups`.

Il ne faut pas supposer qu'une option existe : il faut vérifier les arguments disponibles.

Ajouter des options seulement si elles existent

Pour écrire un code robuste, on peut construire une liste d'arguments puis ne garder que ceux acceptés par la fonction.

```

args_catdes <- list(
  dataset = questionnaire_demo,
  num.var = num_var_catdes,
  generate = FALSE,
  model = "llama3",
  interpretation_mode = "standard",
  isolate.groups = TRUE
)

args_acceptes <- names(formals(NaileR::nail_catdes))

args_catdes <- args_catdes[
  names(args_catdes) %in% args_acceptes
]

res_nail_catdes_options <- do.call(
  NaileR::nail_catdes,
  args_catdes
)

res_nail_catdes_options

```

Ce bloc est utile dans une application pédagogique qui doit fonctionner sur plusieurs machines ou avec plusieurs versions du package.

Appeler un modèle de langage : une étape optionnelle

Dans le cadre du plateau, on travaille surtout avec `generate = FALSE`.

L'appel direct à un modèle de langage doit rester optionnel.

```

# Exemple indicatif seulement.
# À utiliser seulement si le moteur LLM est configuré sur votre machine.

res_nail_catdes_genere <- NaileR::nail_catdes(
  questionnaire_demo,
  num.var = num_var_catdes,
  generate = TRUE,
  model = "llama3"
)

```

Avant d'utiliser `generate = TRUE`, il faut vérifier :

- que le moteur demandé est disponible ;
- que le modèle est installé ou accessible ;
- que le prompt obtenu avec `generate = FALSE` est correct ;
- que les consignes ne demandent pas au modèle d'inventer des résultats.

Comparer méthode manuelle et méthode NaileR

Dans le plateau, les cases précédentes ont montré comment construire un prompt à partir de `catdes()` avec `capture.output()` et `paste()`.

La logique manuelle est instructive, car elle rend visibles les étapes :

```
sortie catdes()
→ capture de la sortie
→ construction du prompt
→ inspection du texte
```

NaileR automatise cette logique.

```
comparaison <- data.frame(
  methode = c(
    "Prompt manuel",
    "Prompt NaileR"
  ),
  geste_R = c(
    "catdes() + capture.output() + paste()",
    "nail_catdes(..., generate = FALSE)"
  ),
  avantage = c(
    "comprendre toutes les étapes",
    "industrialiser la construction du prompt"
  ),
  stringsAsFactors = FALSE
)

comparaison
```

Le prompt manuel sert à comprendre. NaileR sert à stabiliser et accélérer.

Exemple 3 : préparer un artefact textuel

NaileR peut aussi travailler avec des commentaires libres.

On prépare un petit tableau contenant une variable de groupe et une variable textuelle.

```
dataset_textuel_demo <- questionnaire_demo[
  ,
  c(
    "profil_alim",
    "commentaire",
    "satisfaction",
    "intention_achat",
    "prix_percu",
    "naturalite"
  )
]
```



```

)
]

num_var_textuel <- which(
  names(dataset_textuel_demo) == "profil_alim"
)

num_text <- which(
  names(dataset_textuel_demo) == "commentaire"
)

num_var_textuel
num_text

```

Si la fonction `nail_textuel_prep()` est disponible, on peut préparer un artefact textuel sans génération.

```

if ("nail_textuel_prep" %in% getNamespaceExports("NaileR")) {

  res_textuel_prep <- NaileR::nail_textuel_prep(
    dataset = dataset_textuel_demo,
    num.var = num_var_textuel,
    num.text = num_text,
    model = "llama3",
    generate = FALSE
  )

  res_textuel_prep
}

```

L'idée est la même : produire un objet intermédiaire, visible et réutilisable.

Bonnes pratiques

Avant d'utiliser une sortie produite par NaileR, il faut vérifier le prompt ou l'artefact.

Questions à se poser :

- La variable décrite est-elle clairement identifiée ?
- Les groupes ou modalités sont-ils bien définis ?
- Les résultats transmis proviennent-ils de l'objet statistique attendu ?
- Les limites d'interprétation sont-elles explicites ?
- Le prompt interdit-il au modèle d'inventer des résultats ?
- Le mode d'interprétation demandé correspond-il à l'objectif ?

Le bon usage de NaileR n'est donc pas :

`faire générer plus vite`

mais plutôt :

À retenir

NaileR est utile parce qu'il rend reproductible une étape souvent artisanale : transformer des sorties descriptives, factorielles ou textuelles en prompts et artefacts inspectables.

Le workflow recommandé est :

```
produire ou préparer la sortie  
→ inspecter l'objet  
→ construire le prompt ou l'artefact avec generate = FALSE  
→ vérifier le résultat  
→ générer seulement si nécessaire
```

Cette fiche prépare la case **L'Épreuve NaileR** du plateau : l'apprenant dispose maintenant d'un exemple reproductible pour faire tourner NaileR, inspecter ses sorties et comprendre le rôle des principales options.